WALKTHROUGH PREPARATION · SELF-AUDIT

Mistakes, sloppiness and lessons learned

What went wrong, how it was found, what it cost, what was fixed, what is still open -- every finding checked against the code, the evidence or the database.

Berka (PKDD'99) take-home | PostgreSQL | Data quality | Semantic layer | Row level security | LLM as untrusted client

Describes the **delivered version**: branch **delivery-freeze-2026-10-28**, commit **fc0c281** (code b5b18f2), 2026-10-28.
Work done **after** delivery (D028, D029, branch ccr-46f6f45a-t9wtd4) is marked as such wherever it appears.
Every figure comes from the delivered evidence files or a query run on 2026-11-04; nothing is from memory.

Contents

1 How to use this document 3

- What was checked, and how 3
- Status and severity 3
- How to talk about a mistake 3
- The five to raise yourself 4

2 All findings at a glance 5

3 Data and data quality 6

- L-01 · The month-end closing posting 6
- L-02 · The 1,200 CZK "missing posting" was our own artefact 6
- L-03 · The reconciliation rules and the profiler inherited the tie-break 7
- L-04 · The only blocking rule can only catch zero-amount duplicates 7
- L-13 · Q7 rounds before it averages 8
- L-18 · Code hygiene 8

4 Security and privacy 9

- L-05 · A disponent sees more of the owner than promised 9
- L-06 · Hidden values become false values 9
- L-09 · The second-factor lockout can be rolled back 10
- L-10 · The session is bound to the backend PID only 10
- L-11 · sec.v_who_can_reach_client 10
- L-17 · Deployment hardening 10

5 The AI path and the evaluation 12

- L-07 · A guard pass is not a correct answer 12
- L-08 · Why 10/10 is weaker evidence than it sounds 12
- L-12 · "Hand written" 13
- L-16 · The prompt's JSON example had doubled braces 13

6 Documentation and process 14

- L-14 · Documentation drift 14
- L-15 · Mislabels in generated reports 14
- L-19 · Scope against the brief 15
- L-20 · Two working copies 15

7 Bugs found and fixed before delivery 16

8 What was done after delivery 18

- P-01 · The month-end closing posting by balance chain (D028) 18
- P-02 · Intent verification in the runner and the web app (D029) 18
- P-03 · The verifier's first run refused two correct answers 18
- P-04 · Four extra questions 18

9 The lessons, generalised 20

10 The conversation playbook 21

- Questions about mistakes, and how to answer them 21
- Phrases that help 22

1 How to use this document

A walkthrough that defends design choices always reaches the question "what went wrong?". This document prepares that conversation: every mistake, weak spot and sloppy detail found in the delivered version, checked against the code, the evidence files or the database, with its impact, its status and a way to say it.

What was checked, and how

- The **delivered version** is branch `delivery-freeze-2026-10-28`, commit `fc0c281` (code `b5b18f2`). Every finding refers to that version unless it is marked as history or as work after delivery.
- Code claims were read in the delivered tree at the cited line. Data claims were re-run on 2026-11-04 against a database whose `sql/` is identical to the delivered one except for the month-end fact (changed after delivery); figures that depend on the old month-end rule were recomputed with the old rule explicitly.
- Findings already listed in `docs/DELIVERY_AUDIT.md` are marked as **disclosed**. Findings that the delivered documents do not mention, or state wrongly, are marked **not disclosed**. That second group is the one to prepare most carefully.

Status and severity

Status	Meaning
FIXED BEFORE DELIVERY	Found and fixed during the build; a test or a decision record guards it
DISCLOSED	Open in the delivered version, and the delivered documents say so
NOT DISCLOSED	Open in the delivered version, and the delivered documents do not say so — or say the opposite
FIXED AFTER DELIVERY	Fixed on branch `ccr-46f6f45a-t9wtd4` after the freeze; not in what was submitted

Severity	Meaning
High	Changes a delivered number, a stated defect, or a security or correctness claim
Medium	Weakens the evidence behind a claim, or a privacy property for some users
Low	Labelling, wording, hygiene; no number or property changes

How to talk about a mistake

Use the same five beats every time, and keep each to one sentence:

1. **What happened** — plainly, without softening words.
2. **How it was found** — ideally by a check you built, which is itself the point.
3. **What it cost** — measured: which number moved and by how much, who was affected.
4. **What changed** — the fix, or why it is not fixed yet and what fixing it takes.
5. **The lesson** — the general rule you now apply.

An example, the month-end tie-break

"I broke same-day ties by the highest transaction id and documented it as a limitation. When I went back, I found the running balance actually records the order, and the highest id was right only 0.4% of the time. The average barely moved — 1.53 CZK — but 107 accounts' closing balances were wrong by up to 1,200 CZK, and one of my own 'defects' turned out to be that artefact. I fixed it after delivery with the balance chain and a test. The lesson: 'unknowable' is a hypothesis; test it before you document it."

Avoid

Minimising ("it's only 2 CZK"), blaming the tool ("the AI wrote it"), or apologising at length. You directed the build and you checked it; own the result, give the number, give the fix.

The five to raise yourself

Interviewers trust a candidate who names the weak spots before being asked. Raise these during the presentation, at the slide where they belong:

#	Raise it at	Finding
1	The model / Q9	The month-end tie-break, and that the "1,200 CZK missing posting" was its artefact (L-01, L-02)
2	The LLM path	A guard pass is not correctness; the gender query that passed the guard and failed in the database (L-07)
3	The results	Why 10/10 is weaker evidence than it sounds (L-08)
4	Entitlements	The disponent sees the owner's card, and hidden values become false values (L-05, L-06)
5	The opening or the close	Scope against a 1–2 day brief, and what you would cut (L-19)

2 All findings at a glance

ID	Finding	Area	Severity	Status
L-01	Month-end closing posting chosen by highest `trans_id`, wrong for most busy closing days	Data	High	DISCLOSED FIXED AFTER
L-02	The "account off by exactly 1,200 CZK" defect is an artefact of L-01 — and it is in the text the LLM reads	Data / DQ	High	NOT DISCLOSED FIXED AFTER
L-03	Reconciliation rules and profiling inherited L-01: 41 of 55 material breaks are artefacts; "order independent" claim	DQ	High	NOT DISCLOSED FIXED AFTER
L-04	The duplicate rule can only catch zero-amount duplicates; 1,081 repeated interest groups unflagged	DQ	Medium	NOT DISCLOSED
L-05	A disponent sees the owner's card and disposition row — the documents say "nothing else of hers"	Security	Medium	NOT DISCLOSED
L-06	For RLS-restricted users, hidden values become false values (`'no card'` , NULL frequency)	Security / AI	Medium	NOT DISCLOSED
L-07	The guard proves shape, not meaning: the gender query passed it and failed in the database	AI	High	DISCLOSED FIXED AFTER
L-08	The 10/10 evaluation is narrower than it sounds (old pack, question labels in the pack, no hold-out, one sample)	Evaluation	Medium	PARTLY DISCLOSED
L-09	Second-factor lockout can be avoided by a direct client that rolls back failed guesses	Security	Medium	DISCLOSED
L-10	The verified session is bound to the backend PID only; `backend_start` is stored but never compared	Security	Low	PARTLY (A45)
L-11	`sec.v_who_can_reach_client` has a NULL branch that drops grants, and over-states one reach	Security	Low	NOT DISCLOSED
L-12	Reference SQL described as "hand written" / "worked out by a person"	Disclosure	Medium	NOT CORRECTED
L-13	`ai.loan` rounds ages per row, so Q7 status B reads 39.61 instead of 39.60	Data / AI	Low	NOT DISCLOSED
L-14	Documentation drift: stale test counts, figures and diagrams in delivered files	Docs	Low–Medium	PARTLY FIXED
L-15	Mislabels in generated reports (wrong assumption id, reconciliation step, "440 rows", counts that are not rows)	Docs	Low	NOT DISCLOSED
L-16	The prompt's JSON example reached the model with doubled braces `{{ }}`	AI	Low	FIXED AFTER
L-17	Deployment hardening gaps: Docker superuser on an open port, `public` in a SECURITY DEFINER path, owner credentials in the web process	Security	Low–Medium	PARTLY
L-18	Code hygiene: `is_problematic` computed twice; the evaluator ignores `Decimal`	Code	Low	NOT DISCLOSED
L-19	Scope well beyond a 1–2 day brief that warns against infrastructure	Process	Medium	TALK TRACK
L-20	Two working copies diverged; the delivery was reconciled by a partial merge	Process	Medium	DISCLOSED

Bugs found and fixed **before** delivery (chapter 7) and the work done **after** delivery (chapter 8) are listed separately.

3 Data and data quality

L-01 · The month-end closing posting

Area data · **Severity** High · DISCLOSED FIXED AFTER DELIVERY (D028)

What. `core.fact_account_month_end_balance` takes, per account and month, the balance after the latest posting. When several postings share that date, the delivered rule picks the one with the **highest** `trans_id` (D019, A14). The source has no time of day and `trans_id` is not chronological.

Evidence (re-run 2026-11-04). Each posting starts at `balance_after - signed_amount`, so the postings of a day form a chain and its end is the true closing posting. Over 185,057 posted account-months: 154,713 have a unique chain end, 28,041 close on a single posting, 2,303 cannot be ordered. Where the chain resolves, the highest id picks the closing posting **694 times (0.4%)**; the lowest id 96.9%. Interest postings carry high ids but are booked before the month's statement fee, so the old rule usually reads the balance *before* the fee. Example: account 31, November 1998, recorded as 13,312.90; after the 14.60 fee the true closing balance is 13,298.30.

Q9 at 1998-12-31 under ...	Average balance
the delivered rule (highest id)	**43,808.94**
the balance chain, lowest id where ambiguous (D028, after delivery)	43,810.47
the balance chain, highest id where ambiguous	43,810.73
lowest id only	43,808.90

107 accounts close differently at 1998-12-31 under D028 (65 where the chain resolves, 42 where it cannot); the largest difference is 1,200.00 CZK. The maximum balance moves from 138,317.20 to 137,829.50.

Root cause. The design accepted "the source does not record intra-day order" as a fact and documented a tie-break, instead of testing whether the data held indirect evidence of order. Two delivered documents state the stronger, wrong claim that "no derived table can resolve what the source did not record" (`docs/GOLD_LAYER.md`, the deck).

Status. Disclosed at delivery as a known limitation (A14, caveat C2, DDR §3, delivery audit item 3: "Q9's average moves about 2 CZK"). Deliberately not changed in the freeze because it changes a certified number. Fixed after delivery with the chain rule, a `closing_basis` column marking `single posting` / `balance chain` / `ambiguous`, and `tests/test_month_end.py` (the production SQL run on synthetic postings, plus an independent re-derivation over 180,000+ balances with 0 disagreements).

Lesson. "Unknowable" is a hypothesis. Before writing a tie-break down, look for indirect evidence in the data and measure how often the tie-break matters.

> Say it like this
>
> "My month-end rule broke same-day ties by the highest transaction id. I disclosed that as a limitation. Later I found the running balance encodes the order, and the highest id was right only 0.4% of the time. Q9's average moved 1.53 CZK; 107 individual balances moved, by up to 1,200 CZK. I fixed it after submitting, with the chain rule and a test."

L-02 · The 1,200 CZK "missing posting" was our own artefact

Area data quality · **Severity** High · NOT DISCLOSED FIXED AFTER DELIVERY

What. The delivered documents present a showcase finding: *"55 accounts are off by more than 100 CZK and one by exactly 1,200.00 CZK with only 55 postings, which is a missing or extra posting rather than rounding."* That account is **3888**, and the gap is created by L-01.

Evidence. Account 3888, last day 1998-01-31: an interest credit of +1.30 (id 3,467,643, balance 10,500.00 → 10,501.30) and a cash withdrawal of −1,200.00 (id 1,137,901, 10,501.30 → 9,301.30). The chain shows the withdrawal came last; the highest id is the interest posting. The delivered "last reported balance" is therefore 10,501.30, against a net flow of 9,301.30 — a 1,200.00 "gap". Under the chain rule the closing balance is 9,301.30 and the account reconciles exactly.

Where the claim appears in the delivered version. Caveat **C1** in `semantic/semantic_layer.yaml` — and therefore in the context pack the model reads ("one by 1,200 CZK"); the severity rationale of `CON_BALANCE_RECONCILIATION_MATERIAL` in `dq/dq_rules.yaml` and `dq/results/dq_report.md`; `docs/HOW_WE_WORKED_SIMPLY.md`; the deck and its speaker notes ("That is not rounding, that is a posting"); `scripts/make_defect_catalogue.py`.

Root cause. A defect count computed from a derived value inherits the bugs of that derivation. The story was compelling, so it was repeated rather than re-checked.

Lesson. When a number makes a good story, check it twice — and check whether it is a property of the source or of your own logic.

> Say it like this
>
> "The best-sounding defect in my report — an account off by exactly 1,200 — was my own tie-break. On its last day a 1.30 interest credit has a higher id than a 1,200 withdrawal that came after it. Under the chain rule it reconciles to the cent. I like this one because it shows why a derived check can't be trusted more than the derivation it reads."

L-03 · The reconciliation rules and the profiler inherited the tie-break

Area data quality · **Severity** High · NOT DISCLOSED · FIXED AFTER DELIVERY

What. `CON_BALANCE_RECONCILIATION_MATERIAL` / `_ROUNDING` and the profiler's candidates compare the sum of an account's postings with its "last reported balance", taking the last posting by highest id. `dq/results/candidates.md` calls this an "order independent test". It is order dependent exactly where L-01 bites.

Evidence. With the chain rule: material breaks **55 → 14** (the largest now 488.30); rounding band **81 → 70**; accounts reconciling exactly 631 → 637, within 1 CZK 4,364 → 4,416. In the delivered version, 440 warn flags (183 + 114 + 81 + 55 + 6 + 1) fall to 388.

Impact. The data quality report over-states the material defects almost fourfold, and caveat C1 tells the model (and any reader) that 55 accounts cannot be reproduced from their flows. The Q9 reference reports "55 unreconciled" next to the average.

Lesson. A rule that reads a derived value must be tested against an independent derivation, or it reports the derivation's bugs as source defects.

L-04 · The only blocking rule can only catch zero-amount duplicates

Area data quality · **Severity** Medium · NOT DISCLOSED

What. `UNQ_TRANS_BUSINESS_KEY` treats two postings as duplicates when account, date, type, operation, amount **and resulting balance** are equal. A genuine duplicated payment moves the balance, so the second copy has a different `balance_after` and is never caught. By construction the rule catches only duplicates that did not

move the balance: zero-amount postings. The one row it quarantines is a 0.00 CZK posting (account 9051, 1998-02-28).

Evidence. 1,082 groups in staging (1,081 in core, after the quarantine) of identical repeated interest postings — same account, same date, same amount, category `UROK` — worth 157,289.70 CZK beyond the first posting of each group. No rule flags them. Whether they are duplicates or legitimate split interest runs is a business question; it is unverified.

Impact. The delivered narrative uses this rule as the example of a `block` that "double counts a flow and no primary key catches it"; in practice it protected 0.00 CZK.

Fix. A `warn` rule on repeated identical batches (same account, date, amount, category), routed to the data owner; keep the exact-duplicate `block`.

> Say it like this
>
> "My block rule needs the same resulting balance, which means it can only ever catch a duplicate that didn't move money — the one it caught is 0.00. Repeated identical interest postings, a thousand-odd groups worth 157,000 CZK, aren't flagged at all. I'd add a warn rule and ask the bank whether those are duplicates."

L-13 · Q7 rounds before it averages

Area data / AI surface · **Severity** Low · NOT DISCLOSED

`ai.loan` exposes `round(owner_age_at_loan_start, 2)` (`sql/40_ai_views.sql:168`). A model that averages that column averages rounded values: status B reads **39.61** against the exact **39.60** (A 37.07, C 38.70, D 37.00 are unaffected). The evaluator's 0.011 tolerance absorbs it, so the verdict is still `match`. Fix: expose the unrounded value and round in presentation.

L-18 · Code hygiene

Area code · **Severity** Low · NOT DISCLOSED

- `is_problematic` is computed in staging (`sql/10_stg.sql:330`) and again in core (`sql/30_core.sql:344`) with the same predicate. One meaning, two implementations: core should reuse the staged column.
- `evaluate.numeric()` does not treat `Decimal` as a number. Harmless today because rows are converted to floats before evaluation; a trap for anyone who reuses it.

4 Security and privacy

L-05 · A disponent sees more of the owner than promised

Area privacy · **Severity** Medium · NOT DISCLOSED

What. D024 says a disposition grants *the account, never the person*. `docs/ACCESS_AND_AUTH.md` and figure 11 say the disponent "cannot see her client record, or anything else of hers". The client record is correctly hidden; "anything else" is not true.

Evidence (logged in as `client_3764`, 2026-11-04). Client 3764 is a disponent on account 3112, owned by client 3763.

View	What 3764 sees
`ai.client`	only client 3764 — correct
`ai.card`	the owner's card: card 587, *junior*, issued 1996-08-05, `holder_client_id` 3763
`core.bridge_disposition`	both dispositions on the account: 3763 as owner, 3764 as disponent

Root cause. The `entitled_cards` and `entitled_dispositions` policies key on the *account*, which the disponent is entitled to, rather than on the card holder or the disposition's own client. The test checks only `ai.client`.

Fix. Cards visible when the holder is in the caller's clients (or the caller is staff); dispositions visible for the caller's own rows (or staff). Add tests for both, from the disponent's side.

> Say it like this
>
> "I claimed a disposition grants the account and nothing about the owner. That holds for the owner's client record, but the card and disposition policies key on the account, so a disponent sees the owner's card type, issue date and client id. Small, but it's a privacy claim I made too broadly; the fix is a holder-based policy and two tests."

L-06 · Hidden values become false values

Area correctness under RLS · **Severity** Medium · NOT DISCLOSED

What. For the same disponent, `ai.loan` returns loan 5610 with `owner_card_type = 'no card'`, `statement_frequency` NULL and `owner_gender` NULL. The truth is *junior* and *monthly*. The owner join returns nothing under RLS (the owner's client row is invisible), and the view's `coalesce(owner_card_type, 'no card')` (`sql/40_ai_views.sql:169`) turns "not visible to you" into a confident wrong value.

Impact. Q4 or Q10 asked by a disponent is *wrong*, not merely *restricted*. The semantic layer does not tell the model that a NULL can mean "hidden". This is the most interesting finding for an AI audience: entitlement filtering changed the meaning of a column.

Fix. Never coalesce over a join that RLS can empty; expose an explicit "not visible" value or leave NULL, and describe that in the semantic layer; test the pre-joined views as each persona.

Say it like this

"Row level security can change the meaning of a column, not just the number of rows. A disponent asking my Q4 gets 'no card' for a loan whose owner has a junior card, because the owner is hidden and the view coalesces the missing value. Hidden is not the same as absent, and the view should never say otherwise."

L-09 · The second-factor lockout can be rolled back

Area authentication · **Severity** Medium · DISCLOSED

The failure counter is written inside the caller's transaction. The project's own client commits every attempt, so five failures lock the factor for 15 minutes (tested since `b5b18f2`). But anyone holding a principal's password can connect directly, roll each failed guess back, and is never locked out. Found by the delivery audit when the lockout test was finally written; recorded as a strict expected failure so the test turns red on the day it is fixed; disclosed in ACCESS_AND_AUTH, the demo script, the visual guide, the README and the DDR. Row level security is unaffected: without the second factor a principal still reads no rows. Fix: count attempts outside the caller's transaction (an autonomous write via a separate connection or `dblink`), or accept consumer logins only through a gateway.

Lesson. A documented control without a test is a hope. The lockout was documented for weeks before a test existed; writing the test found the gap.

L-10 · The session is bound to the backend PID only

Area authentication · **Severity** Low · PARTLY DISCLOSED (A45)

`sec.has_verified_session()` matches `db_role` and `pg_backend_pid()`; `backend_start` is stored when the session is opened but never compared (`sql/51_sec_auth.sql:205`, `:303-313`). If a PID is reused by the same role within the 30-minute window, the new connection inherits the session. A45 names the PID-reuse risk; the unused column is the cheap fix.

L-11 · `sec.v_who_can_reach_client`

Area access reporting · **Severity** Low (latent) · NOT DISCLOSED

The view that answers "who can reach this client, and why" has a branch with `g.db_role <> (SELECT … LIMIT 1)`: for a client without a login the subquery is NULL, the comparison is NULL, and indirect grants silently drop out. `u.db_role <> ''` is a no-op (`sql/51_sec_auth.sql:570-572`). It is latent with today's seed (every relevant client has a login) and it also over-states one reach (it lists `client_3763` as able to view and operate client 3764's record). Fix: `IS DISTINCT FROM` or `NOT EXISTS`, and a test with a client that has no login.

L-17 · Deployment hardening

Area operations · **Severity** Low–Medium · PARTLY DISCLOSED

Gap	Where	Note
The Docker path makes `berka_owner` the image's superuser and publishes 5432 on all interfaces	`docker-compose.yml:15-18`	`docs/DEPLOYMENT.md` says not to open 5432; bind to 127.0.0.1
A SECURITY DEFINER function's `search_path` puts `public` before `pg_catalog`	`sql/51_sec_auth.sql:230`	Low risk where consumers cannot create in `public`; schema-qualify the pgcrypto calls

Gap	Where	Note
The web process holds owner credentials for the fixed tour pages	`src/webapp.py`	A code-execution bug there is owner compromise; pre-render those pages instead
TOTP secrets stored recoverable	`sec.auth_factor`	A44; encrypt with a key outside the database
Demo prints codes; personas share a demo password	`/api/code`, console	A46; demo affordances, labelled

5 The AI path and the evaluation

L-07 · A guard pass is not a correct answer

Area AI · **Severity** High · DISCLOSED · FIXED AFTER DELIVERY (D029)

What. The guard parses the model's SQL and enforces its shape; it does not resolve columns or judge meaning. In the delivered version the intent checker is a prototype (`scripts/ir_check_prototype.py`) not imported by the application. Disclosed as delivery audit item 2 and in the DDR.

Evidence — the observed failure. *"Find male or female loans in Q1 1995."* The model joined `ai.loan` to `ai.account_owner` and used `T2.gender` . The guard allowed it; PostgreSQL raised `column t2.gender does not exist` . Gender lives on `ai.client` ; `ai.loan` and `ai.account_owner` expose `owner_gender` . The semantic layer's `gender_share` metric had said "any entity with a gender column"; the delivery corrected that text and made the validator refuse a metric naming a missing column, but did not verify meaning. The failure was safe — no rows, an error recorded — but the same class of mistake with a column that *does* exist (a wrong date, the wrong quarter, a fan-out join) would have returned a plausible wrong number. The local model's Q6 (a fan-out to every calendar day: 279 "new accounts" in January instead of 106) is exactly that.

Fix (after delivery). D029: the model returns an intent (IR) with its SQL; `src/sql_ir.py` verifies IR and SQL together before any session is opened, then plans, executes and recomputes the certified metric from the IR; rows only when both agree. Active in the runner and the web app.

> Say it like this
>
> "The guard is a shape filter, not a correctness check. The gender question proved it: one safe SELECT, allowed, then rejected by the database for a column that doesn't exist. That one failed safely — the dangerous version is the query that is valid and wrong. That's why I added intent verification after submitting."

L-08 · Why 10/10 is weaker evidence than it sounds

Area evaluation · **Severity** Medium · PARTLY DISCLOSED

Weakness	Status in the delivery
The recorded Claude run (2026-10-25, `a7da25a`) used the 29,500-character pack from **before** the semantic audit; the delivered pack is 33,662 characters and was not re-run with Claude	Disclosed (P1 item 1)
The pack names the questions: "Relevant to: Q1", "Answers Q2, Q3, Q4, Q7 and Q10 with no further joins" (19 lines carry a question id) — it shows the model where to look, without numbers	Not disclosed
"Q1" in the pack means a reference question; in a user's question it means a quarter (the audit added QR11 to disambiguate)	Addressed by the audit
No hold-out set: no unseen question was evaluated	Not disclosed
One sample per question; one key measure per question; order checked for three questions	Not disclosed as a limitation
The same build session wrote the semantic layer and the reference SQL	See L-12

After delivery. Claude was re-run on the new pack (10/10, with the verifier active) and four new questions — gender of loans and calendar quarters — were added as a separate suite (4/4).

Lesson. An evaluation tuned to its own test set measures the author, not the system. Keep a hold-out set and keep question ids out of the context.

L-12 · "Hand written"

Area disclosure · **Severity** Medium · NOT CORRECTED

`AI_DISCLOSURE.md` labels `sql/60_reference_queries.sql` "Hand written. Ground truth."; the plain-words document says "a person worked out all ten answers the slow, careful way"; the deck says "the answers a person had worked out by hand beforehand". The same disclosure says the pipeline, rules, semantic layer and tests were written with Claude in an interactive session — and so was the reference SQL. What is true and sufficient: the reference SQL was **not generated by the model under test**, never entered its context, and every reference answer was independently **re-derived from the raw text tables** and matched.

> Say it like this
>
> "To be precise about authorship: I built everything with an AI coding assistant under my direction, including the reference SQL. 'Hand written' in my disclosure means not written by the model under test — and I re-derived every reference answer from the raw text with separate SQL, so the model and the reference cannot be wrong in the same way."

L-16 · The prompt's JSON example had doubled braces

Area AI · **Severity** Low · FIXED AFTER DELIVERY

`semantic/prompt_template.md` writes its example JSON with `{{` and `}}` (as if for Python's `format`), but the runner fills the template with `str.replace`, so the model received `{{ "sql": … }}` literally — an invalid JSON example. Strong models ignored it; it is still a defect in the one artefact the model reads besides the pack.

6 Documentation and process

L-14 · Documentation drift

Area documents · **Severity** Low–Medium (credibility) · PARTLY FIXED AT DELIVERY

The delivery freeze rewrote the README as an index, moved five dated reviews to `docs/history/` and merged the walkthrough notes into the interview pack, because they carried stale figures. Some stale figures survived in delivered files:

Where (delivered)	Says	Current
`docs/DEPLOYMENT.md:114`	"202 tests at the end"	327 passed, 1 skipped (330 collected)
`deliverables/Walkthrough_Deck.pptx` (not a deliverable, but the walkthrough deck)	"137 tests", "make clean all – four and a half minutes", "55 accounts … one by exactly 1,200", "10 of 10 matched the answers a person had worked out by hand"	See L-02, L-12; build time measured once on Linux
VISUAL_GUIDE figure 9 (system map)	PostgreSQL 16 only; 88 rules / 123 executed; `rm_south` 1,386 / 1,157 / 194; 137 tests; 4m28s; workbook 9 tabs	89 / 124; 1,387 / 1,158 / 195; 11 tabs; PostgreSQL 16 and 18
VISUAL_GUIDE figure 11 (access model)	the disponent "cannot see … anything else of hers"	See L-05
`docs/GOLD_LAYER.md`, deck	"no derived table can resolve what the source did not record"	See L-01
`docs/INTERVIEW_PREPARATION.md`	PostgreSQL 16; 241 tests	Superseded by this study pack
Context-pack size	33,500 (AI disclosure), 33,700 (delivery audit)	33,662 characters measured
README, DDR, model explanation	"4 dimensions"	4 dimensions from source tables + a generated calendar = 5 `dim_` tables

Test counts over the life of the project: 114 → 137 → 202 → 241 → 245 → 327 passed (+1 skipped, 330 collected at delivery). Each was true when written; the problem is that hand-typed numbers do not update themselves.

Lesson. Every figure in a document should be generated by the command that measures it, or live in exactly one document that the others link to. The delivery audit is that document now.

L-15 · Mislabels in generated reports

Area generated documents · **Severity** Low · NOT DISCLOSED

- **Cleaning ledger C05** says "the century is an assumption (A25)"; the century assumption is **A17** (A25 is the placeholder list). The reference is hard-coded in `src/cleaning_ledger.py:141`.
- **Cleaning ledger C21** is titled "Prove core = staging − quarantine, on rows and on money", but its probe counts the **11 raw-vs-staging checks** of the DQ run (`dq.reconciliation`, where `core_value` is empty). The core = staging − quarantine check is `build_model.reconcile_core` (9 checks). Both balance, so no number is wrong — the label is.

- **"440 flagged rows"** (README, DDR) are 440 *flags* on **352 distinct rows** (the ledger's C15 says 352). One row can carry several warn flags.
- **Info "rows affected 2,556,194"** (DQ report) and **"1,073,386"** (defect register, missing values) count the same missing `k_symbol` rows once per rule or per column (code and label); they are counts of findings, not of rows.
- **The defect register's example column** is sampled without an `ORDER BY`, so it can change between runs (disclosed, delivery audit item 9).

L-19 · Scope against the brief

Area process · **Severity** Medium (perception) · TALK TRACK

The brief expects 1–2 days and warns: *"If you spend most of your time on infrastructure or tooling setup, you have misread the brief."* The repository contains a two-factor login enforced in the database, a web walkthrough, a deployment guide for a public host, a 55-slide deck, simulated layer-owning agents and a native-Windows/Ollama path. An interviewer will ask.

How to answer. (1) Tasks 1–3 were built and evaluated first, and the stack stayed deliberately minimal — one database, SQL files, plain Python, a Makefile. (2) The extras were added on explicit request and each makes a *governance* point rather than an infrastructure one: identity enforced in the engine, onboarding as data, a model that runs fully offline. (3) What you would cut, in order: the deployment guide, the simulated agents, the web page (keep the console demo). Do not deny the time spent and do not apologise for it.

L-20 · Two working copies

Area process · **Severity** Medium · DISCLOSED (DELIVERY AUDIT §1)

The Windows/Ollama work existed only as **uncommitted changes on the laptop**, while the cloud sessions pushed the semantic audit and, later, the month-end correction and the integrated verifier. At delivery the laptop work was committed first (plus a backup branch), then origin was merged **up to `c855be4` only**, deliberately leaving out the commits that change Q9. Nothing was lost, and the reason for the partial merge is written down. The database used for the delivery evidence was built from an earlier commit (`5b13f2b`) and not rebuilt; the audit argues, with the diff, that the build code is unchanged since.

Lesson. Commit and push from every working copy every day. A partial merge under deadline is manageable when it is deliberate and recorded — but it should never be the plan.

7 Bugs found and fixed before delivery

These are the stories that show the checks working. Each was caught by a check the project built, and each has a test or a decision record guarding it.

ID	What happened	How it was found	Guard now
H-01	**The rule self-test nearly deleted real data.** `dq.v_blocked` was not scoped by schema, so injected defects excluded *real* rows: 76 districts instead of 77, 11,024 transactions orphaned	The model's foreign-key and grain checks failed before any deliverable was written	Views filter `target_schema = 'stg'`; D008
H-02	**The entitlement filter leaked through a view.** Three curated `core` views were not `security_invoker`: `ai.loan` returned all 682 loans to a user entitled to none, while `core.fact_loan` returned 0	Logging in as a restricted user and counting rows	`test_every_view_between_the_consumer_and_the_policy_is_security_invoker`; D010
H-03	**Rebuilding core dropped the AI surface.** A single `make model` re-run left the `ai` views missing; the LLM retried after `relation "ai.client" does not exist`	The model's own retry log	`make model` restores views, policies and grants; D012
H-04	**Materialised grants drifted.** Granting a portfolio inserted a membership row and no grant, so the access did nothing until the next rebuild	An onboarding test	Grants derived live; D027
H-05	**The demo's own query was wrong.** The worked example totalled the loan book with `sum(DISTINCT amount)` as a reflex guard against fan-out; there was no fan-out, and DISTINCT dropped the 37 loans that share an amount with another: 98,780,760 instead of 103,261,740 (−4,480,920 CZK)	The build review	`test_the_demo_question_totals_correctly_for_an_unrestricted_caller` pins the total to the base table
H-06	**An unknown code shipped unlabelled.** `LEASING` on 341 standing orders was not in the code map, and the check did not cover that table	Extending the code check to every coded column	Build fails on an unlabelled code; D017, A28
H-07	**The cleaning ledger triple-counted** on a re-run	Re-running the ledger	`tests/test_dq.py`
H-08	**The worked example reported 0 accounts for a disponent**	Reading its output against the grant model	`tests/test_rls.py`
H-09	**The evaluator was too loose and too strict.** A relative tolerance of 1e-4 would have accepted the top Q5 district wrong by 11,513 CZK; and Q6 scored `mismatch` with identical counts because it looked for `1` where the model wrote `1996-01`	Reviewing the first evaluation	Absolute tolerance 0.011; any published representation of a dimension accepted; the change is written down in D011
H-10	**An assumption's "what if wrong" was not measured.** A10 claimed a calendar-year age difference reads 0.5 years high; measured, it is within 0.11 — it is *completed* years that read 0.5 low	Re-measuring every stated alternative	A10 corrected 2026-10-28
H-11	**Growing the semantic layer broke the local model.** The audit grew the pack to about 33,700 characters and Ollama's pre-check refused every question at a 12,288-token window: 13 tests failed after the merge	The test suite	Window raised to 16,384; a test keeps the window in step with the real prompt

ID	What happened	How it was found	Guard now
H-12	**Windows-only failures.** `password authentication failed` for the owner (init-db now always resets the password); ZIP entry names written with backslashes; `psql` looked up at a hard-coded laptop path	Running natively on Windows	Fixed in `scripts/windows.ps1` and `src/make_zip.py`
H-13	**The DQ report labelled its rule count "defect types"**	The delivery audit	Report now says it counts rules
H-14	**Clock drift produced an already-spent TOTP code**; and the test suite locked itself out by accumulating failures across runs	Intermittent auth test failures	Fresh-window code helper; `tests/conftest.py` resets counters (a workaround, not a test — L-09 shows why that matters)

Two of these make the best stories

"Twice my own checks caught me. The rule self-test nearly removed real data — the build failed on a grain check before anything was written out. And the entitlement filter leaked through one view missing a setting — found only because the test logs in as a restricted user and counts. Both are written down rather than quietly fixed, because a system that has never caught itself being wrong hasn't been tested."

8 What was done after delivery

All on branch `ccr-46f6f45a-t9wtd4` (commits `49059c0`, `a308fc1`, `6943873`, 2026-10-28), **not** merged into the delivery freeze. 321 tests passed on that branch.

P-01 · The month-end closing posting by balance chain (D028)

The closing posting of a day is the end of the running-balance chain; where the chain cannot order the postings, the lowest id among the chain ends is the documented fallback and the row is marked `closing_basis = 'ambiguous'`. The reconciliation rules read the closing balance the same way.

	Delivered	After D028
Q9 average balance at 1998-12-31	43,808.94	43,810.47
Accounts with a different closing balance at book end	—	107 (max 1,200.00)
Material balance breaks (> 100 CZK)	55	14 (max 488.30)
Rounding-band breaks (1–100 CZK)	81	70
Accounts at book end whose closing day is ambiguous	(not measured)	42 of 183

Evidence: `tests/test_month_end.py` (the production SQL block on synthetic postings: same-day chains, gaps, first and last months, ties, a cycle, a zero-amount posting; and an independent head-of-chain derivation over 180,000+ real balances with 0 disagreements). The earlier evidence is archived, not overwritten (`dq/results/archive/2026-10-28_before_D028/`).

P-02 · Intent verification in the runner and the web app (D029)

The model returns an IR next to its SQL; `src/sql_ir.py` checks the IR against the certified metrics, resolves every column scope by scope, evaluates the period filter day by day over the book's calendar, refuses undeclared or fan-out joins, unrequested filters and pivots, and only then logs in, plans (EXPLAIN), executes under RLS and recomputes the metric from the IR. One entry point (`run_questions.verify_and_execute`) serves both paths; a refusal is fed back for the one retry. `tests/test_sql_ir.py` (66 tests) covers the observed gender failure, quarter inclusion and exclusion, the wrong date column, fan-out, missing periods and grouping, malformed IRs, and both application paths with a scripted model.

P-03 · The verifier's first run refused two correct answers

On its first live run the verifier refused Q9 (the model wrote the composite grain as the string `"account_id, month_end_date"` instead of a list) and Q6 on the first attempt (months reported as numbers, which the recomputation did not accept). Both answers were right; both were verifier bugs. They were fixed and pinned by tests, that run was archived with an explanation, and both suites were re-run on the fixed code: **10/10** and **4/4**, all on the first attempt.

Lesson. A checker needs tests against *correct* answers as much as against wrong ones; otherwise it trades false answers for false refusals.

P-04 · Four extra questions

A separate suite (`data/questions_extra.csv`, `sql/61_reference_extra.sql`, independent SQL on core tables) for the failure classes the ten did not cover:

ID	Question	Reference	Claude
E1	Find male or female loans in Q1 1995	18 male-owned, 10 female-owned (28 loans; A47)	match
E2	Total loans granted in Q1–Q3 1995 vs Q1–Q3 1998	9,033,564 vs 21,046,932	match
E3	New accounts opened in Q1 1996	334	match
E4	Loans granted per quarter in 1996	20, 23, 39, 35	match

A47 (loans by the owner's gender, not loan–client pairs: 34 pairs because 6 loans have a disponent of the other gender), A48 (calendar quarters, one row per period) and A49 (interest counts as incoming) are recorded as **awaiting business confirmation**.

9 The lessons, generalised

Twelve rules that the findings above taught. These are what an interviewer is really asking for when they say "what did you learn?" — each comes with the finding that taught it.

#	Lesson	Taught by
1	**A derived check inherits the bugs of the derivation it reads.** Test it against an independent derivation	L-02, L-03
2	**"Unknowable" is a hypothesis.** Look for indirect evidence in the data before documenting a tie-break; measure how often it matters	L-01
3	**A green build is not a correct build.** Verify the model's grains, keys and reconciliation after every build	H-01, H-03
4	**Test security as the attacker.** Log in as the restricted user and count rows; never trust reading your own DDL	H-02, L-05
5	**Safe and correct are different properties.** A query can be harmless and wrong; governance must make wrong answers detectable, not only harmful queries impossible	L-07
6	**Hidden is not absent.** Never let a view turn "not visible to you" into a value	L-06
7	**Do not tune the evaluation to the test.** Keep a hold-out set, keep question ids out of the context, sample more than once	L-08
8	**Hand-typed numbers drift.** Generate every figure from the command that measures it, or keep it in one document the others link to	L-14
9	**Precise disclosure beats generous disclosure.** Say exactly who wrote what; overstatement costs more credibility than the fact	L-12
10	**A documented control without a test is a hope.** Writing the lockout test found the rollback gap	L-09
11	**A checker needs tests against correct answers too**, or it trades false answers for false refusals	P-03
12	**Commit and push from every working copy, every day**, and scope to the brief — demonstrate governance, not infrastructure	L-19, L-20

10 The conversation playbook

Questions about mistakes, and how to answer them

1 What was your biggest mistake?

The month-end tie-break. I accepted that the source had no intra-day order and documented a rule instead of testing it; the running balance did encode the order. It was right only 0.4% of the time, it produced a fake "1,200 CZK missing posting" in my own defect report, and it inflated my material reconciliation breaks from 14 to 55. The average was robust; the account-level balances were not. I fixed it after delivery with the chain rule and a test. (L-01–L-03)

2 What would you do differently?

Three things. Test every "the data cannot tell us" claim against the data before writing it down. Verify the meaning of generated SQL, not only its shape, from the start. And keep a hold-out set of questions so the evaluation measures generalisation, not my own semantic layer.

3 Did you find anything wrong after you submitted?

Yes, and I prefer to tell you than to have you find it: the tie-break artefacts, the disponent seeing the owner's card, hidden values shown as "no card", a duplicate rule that can only catch zero-amount duplicates, and a few mislabels in generated reports. Two of them I fixed on a branch, with before/after evidence; the rest are listed with their fix.

4 Why didn't you merge the fix into the submission?

It changes a certified number and its reference answer. Changing a certified number during a delivery freeze, without the business signing off the new rule, is exactly what a governed product should not do. So the submission discloses the limitation, and the fix waits on a branch with its tests.

5 How much of this did the AI write?

Most of the code, under my direction, with every step executed and checked against the database; the disclosure names each model and role. The model under test never saw the reference SQL or the data. One wording in my disclosure is too generous — "hand written" — and the precise version is "not written by the model under test, and independently re-derived from raw". (L-12)

6 If the guard does not check meaning, why should anyone trust an answer?

In the delivered version, because the semantic layer removes most reasons to guess, a wrong name fails safely in the database, and the evaluation compares values with independent SQL. That is not enough for live users, which is why the next step — built after delivery — verifies the model's declared intent against its SQL and recomputes the certified metric before any number is shown. (L-07)

7 Isn't this over-engineered for a 1–2 day take-home?

The extras are, and I can say what I would cut first: the deployment guide, the simulated agents, then the web page. The core stayed deliberately small — one database, SQL files, Python — so that the time went into the data, the definitions and the enforcement, which is what the brief is about. (L-19)

8 What are you most proud of?

That the system caught me twice — the self-test that nearly deleted real data and the view that leaked — because I built checks that distrust the build. And that the entitlement filter is somewhere the model's text cannot reach.

Phrases that help

Situation	Say
Admitting a defect	"That is a real defect. Here is what it changes, by how much, and the fix."
A number you do not have	"I did not measure that. The closest figure I have is ..., and I would measure it with ..."
A claim that was too broad	"I stated that too broadly. What is true is ..."
A design choice under attack	"The alternative was ...; I chose this because ...; the cost is ..."
Delivered vs later work	"In what I submitted ... ; after submitting I ..."
Pressure to agree with a wrong premise	"I don't think that is quite right — the evidence is ..."

Never say

- "The guard guarantees the SQL is correct."
- "No derived table can resolve what the source did not record."
- "A person wrote all the reference answers by hand."
- "The disponent cannot see anything of the owner's."
- "Q9 is 43,810.47" — as the delivered answer. Delivered: 43,808.94.
- "There are 241 tests." Delivered: 327 passed, 1 skipped.